```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
#2748  365   OUI
#2748  365O  NON
def verif(ch) :
    return len(ch)>0 and ch.isdecimal()
def posmin(ch):
    p=0
    for i in range(1,len(ch)):
        if ch[i]<ch[p]:
            p=i
    return p
def Verifier(m,n) :
    ch=str(m) + str(n)
    l=len(ch)
    chtrie=''
    while ch!='' :
        p=posmin(ch)
        chtrie=chtrie+ch[p]
        ch=ch[:p] + ch[p+1:]
    i=0
    while i<l-1 and int(chtrie[i+1]) - int(chtrie[i])==1:
        i+=1
    res=str(m)+" et "+str(n)
    if i==l-1:
        res=res+" forment une succession parfaite "+'('+chtrie+')'
    else :
        res=res+" ne forment pas une succession parfaite "+'('+chtrie+')'
    return res
```

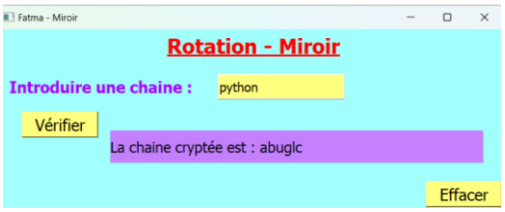




```
def Play():
    chm=w.m.text()
    chn=w.n.text()
    if not verif(chm) or not verif(chn) :
        Msg='Veuillez introduire 2 entiers positifs'
    else :
        Msg=Verifier(int(chm),int(chn))
    w.res.setText(Msg)
def Reset():
    w.m.clear()
    w.n.clear()
    w.res.clear()

x=QApplication([])
w=loadUi("Interfacesuccession.ui")
w.verif.clicked.connect(Play)
w.vid.clicked.connect(Reset)
w.show()
x.exec()
```

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
def Alpha(ch):
    i=0
    l=len(ch)
    ok = 0<l<10
    while i<l and ok :
        if "a"<=ch[i]<="z":
            i=i+1
        else:
            ok=False
    return ok
def rotation(ch):
    res=''
    for i in range(len(ch)):
        res=res+chr(97+(ord(ch[i]) - 97 +13)%26)
    print(res)
    return res
def miroir(ch):
    res=''
    for i in range(len(ch)):
        res=ch[i]+res
    return res
def Play():
    ch=win.mot.text()
    if ch=='':
        mess="Veuillez introduire une chaine"
    elif not Alpha(ch):
        mess="Veuillez introduire une chaine valide !"
    else:
        mess='La chaine cryptée est : ' + miroir(rotation(ch))
    win.r.setText(mess)
```



```
def Effacer():
    win.mot.clear()
    win.r.clear()

from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
app=QApplication([])
win=loadUi("Interfacerotationmiroir.ui")
win.show()
win.occ.clicked.connect(Play)
win.eff.clicked.connect(Effacer)
app.exec_()
```




```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
#la vie est une aventure merveilleuse
def Alpha(ch):
    i=0
    ok=True
    l=len(ch)
    while i<l and ok :
        if "a"<=ch[i]<="z" or ch[i]==" ":
            i=i+1
        else:
            ok=False
    return 0<l<50 and ok and ch[0]!="" and ch[l-1]!=" "
def Invers(ch):
    res=''
    for i in range(len(ch)):
        res=ch[i]+res
    return res
def Miroir(ch):
    res=''
    ch=ch + " "
    while ch!='':
        p=ch.find(' ')
        res=res+Invers(ch[:p])+' '
        ch=ch[p+1 :]
    return res
```

```
def Play():
    ch=win.mot.text()
    if ch==":
        mess="Veuillez introduire une chaine"
    elif ch.find("  ")!=-1 :
        mess="Entre 2 mots un seul espace est autorisé"
    elif not Alpha(ch):
        mess="Veuillez introduire une chaine valide"
    else:
        mess=Miroir(ch)
    win.res.setText(mess)
def Effacer():
    win.mot.clear()
    win.res.clear()

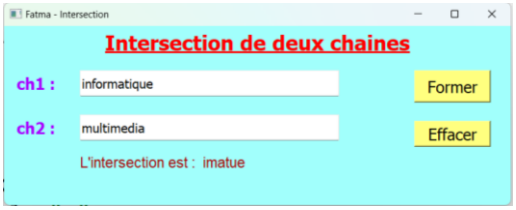
app=QApplication([])
win=loadUi("InterfaceMiroirsMots.ui")
win.show()
win.mir.clicked.connect(Play)
win.eff.clicked.connect(Effacer)
app.exec_()
```

```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
from numpy import array
#Une hirondelle ne fait pas le printemps.
def NBMot(ch):
    nb=1
    for i in range(len(ch)):
        if ch[i]==' ':
            nb=nb+1
    return nb
def Trier(ch):
    nb=NBMot(ch)
    t=array([str]*nb)
    for i in range(nb-1):  #Transfert de la chaine vers un tableau
        p=ch.find(' ')
        t[i]=ch[:p]
        ch=ch[p+1 :]
    t[nb-1]=ch[:len(ch)-1]   #supprimer le point
    print(t)
    for i in range(nb-1):
        pcourt=i
        for j in range(i+1,nb):
            if len(t[j])<len(t[pcourt]):
                pcourt=j
        if pcourt!=i:
            aux=t[pcourt]
            t[pcourt]=t[i]
            t[i]=aux
    print(t)
```



```
    ch='' # Reconstitution de la chaine de plus court au plus long
    for i in range (nb) :
        ch= ch+t[i]+' '
    return ch
def Play():
    ch=win.ph.text()
    if ch=='':
        mess="Veuillez introduire une phrase"
    elif ch.find("  ")!=-1 :
        mess="Entre 2 mots un seul espace est autorisé"
    elif ch[len(ch)-1]!='.' :
        mess="La chaine doit se terminer par un point"
    elif not "A"<=ch[0].upper()<="Z":
        mess="La chaine doit commencer par une lettre"
    elif NBMot(ch)>15 :
        mess="La chaine doit contenir au maximum 15 mots"
    else:
        mess=Trier(ch)
    win.r.setText(mess)
def Effacer():
    win.ph.clear()
    win.r.clear()

app=QApplication([])
win=loadUi("InterfaceTirage.ui")
win.show()
win.tri.clicked.connect(Play)
win.eff.clicked.connect(Effacer)
app.exec_()
```




```
from PyQt5.uic import loadUi
from PyQt5.QtWidgets import QApplication
from numpy import array
#informatique  multimedia
def Alpha(ch):
    i=0
    ok=True
    l=len(ch)
    while i<l and ok :
        if "a"<=ch[i]<="z":
            i=i+1
        else:
            ok=False
    return 0<l<30 and ok
def Recherche(ch1,ch2):
    l=len(ch1)
    res=" "
    for i in range(l):
      if ch2.find(ch1[i])!=-1 and res.find(ch1[i])==-1:
          res=res+ch1[i]
    return res
def Play():
    ch1=win.ch1.text()
    ch2=win.ch2.text()
    if ch1=='' or ch2=='':
        mess="Veuillez introduire deux chaines non vides"
    elif not Alpha(ch1) or not Alpha(ch2) :
        mess="Veuillez introduire deux chaines valide"
    else:
        mess="L'intersection est : "+Recherche(ch1,ch2)
    win.res.setText(mess)
```

```
def Effacer():
    win.ch1.clear()
    win.ch2.clear()
    win.res.clear()

app=QApplication([])
win=loadUi("InterfaceIntersection.ui")
win.show()
win.form.clicked.connect(Play)
win.eff.clicked.connect(Effacer)
app.exec_()
```